



UFC

**UNIVERSIDADE FEDERAL DO CEARÁ
CAMPUS QUIXADÁ
INSTRUMENTAÇÃO**

**ANA LÍVIA DA SILVEIRA LOPES - 514380
ARTHUR ROBERTO DA SILVA - 536330
PEDRO NÓBREGA DAMACENA - 536543
WILLIAM MARREIRO BRITO - 538468**

REDE DE SENSORES SEM FIO

**QUIXADÁ
2026**

1. Introdução

Este trabalho descreve o desenvolvimento de uma Rede de Sensores Sem Fio (WSN) voltada para o monitoramento ambiental de uma sala de aula. A motivação para o desenvolvimento deste projeto reside na necessidade de otimizar o conforto térmico e a eficiência energética em ambientes compartilhados. A solução proposta utiliza uma topologia centralizada, onde nós sensores de baixo consumo energético comunicam-se com um Gateway, responsável pela agregação dos dados e pela interface com plataformas de visualização e armazenamento em nuvem.

Os objetivos do projeto são montar e programar uma WSN de seis nós, coletar e registrar dados de temperatura, umidade e luminosidade interna e temperatura externa, e desenvolver a interface de controle por IrDA do Ar-Condicionado.

Para alcançar tais objetivos, foi definida uma série de etapas metodológicas:

- **Integração de Sensores:** Conexão do sensor AHT10 (via I²C) e do LDR ao microcontrolador STM32, com a devida implementação das rotinas de leitura de luminosidade, temperatura e umidade.
- **Comunicação Sem Fio:** Configuração do módulo nRF24L01 via SPI no STM32 e programação do envio dos dados coletados para o nó central.
- **Desenvolvimento do Gateway:** Configuração do ESP32 como Gateway e implementação do emissor infravermelho (IR) para o controle automatizado dos aparelhos de ar-condicionado.
- **Conectividade e Dados:** Implementação do protocolo MQTT para o envio dos dados a uma API externa e conexão com a *stack* de monitoramento composta por InfluxDB e Grafana.
- **Hardware e Energia:** Montagem das PCBs, integração das baterias Li-Po 3.7V com módulos reguladores de tensão e verificação da autonomia e eficiência do sistema de alimentação.
- **Otimização e Finalização:** Adição de rotinas de *DeepSleep* para economia de energia, documentação da configuração de hardware (pinos, sensores e transceptores) e realização de testes finais no sistema completo de coleta e comunicação.

2. Arquitetura de Hardware

A infraestrutura de hardware da rede foi dividida em duas categorias de dispositivos, otimizadas para suas funções específicas: os Nós Sensores e o *Gateway*.

2.1 Nós Sensores

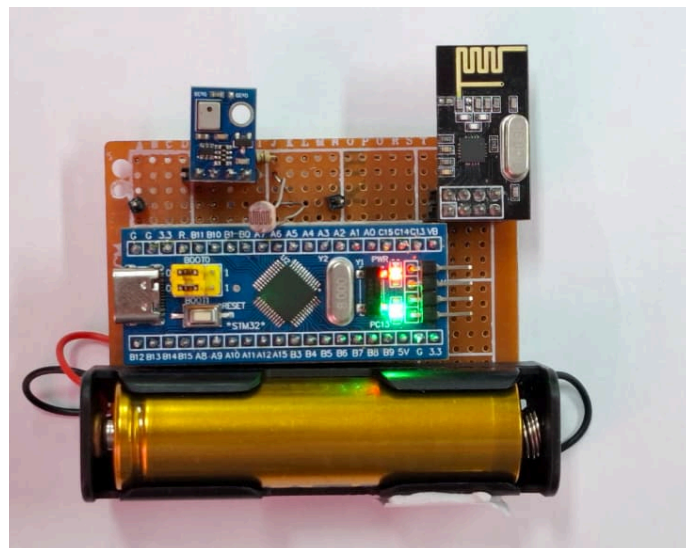
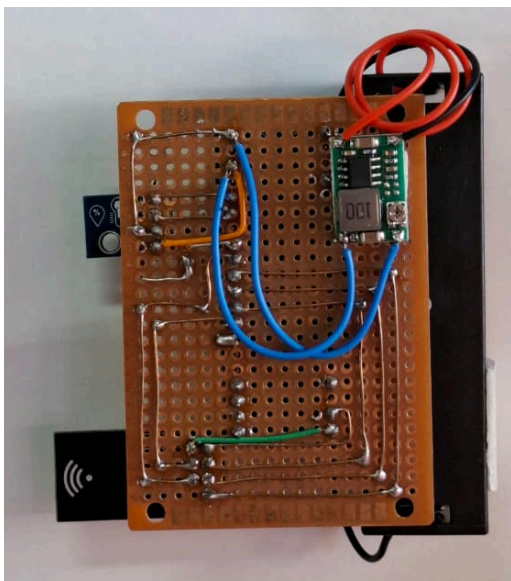
Os nós sensores foram construídos com base no microcontrolador **STM32**, selecionado por sua arquitetura ARM Cortex-M eficiente, ideal para aplicações dependentes de bateria. Programado via STM32CubeIDE. Cada nó integra os seguintes periféricos:

- **Sensor AHT10:** Responsável pela leitura de temperatura e umidade, interfaceado via barramento I²C;
- **LDR (Light Dependent Resistor):** Utilizado para o monitoramento da luminosidade ambiente, conectado a uma entrada analógica (ADC) do microcontrolador.

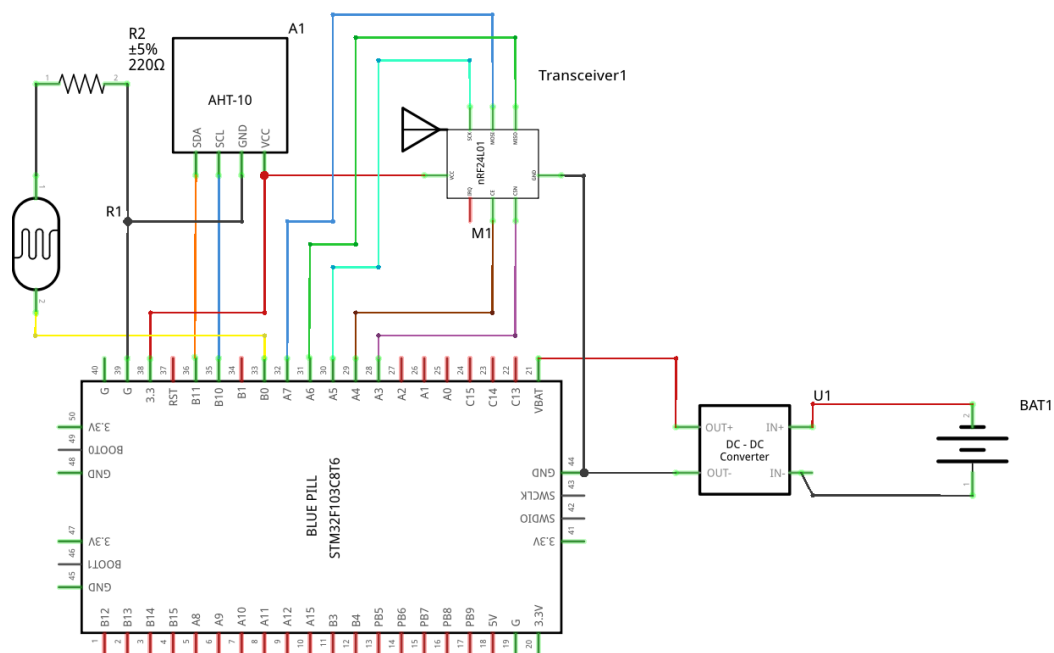
- **Transceptor nRF24L01:** Módulo de rádio frequência operando em 2.4 GHz, conectado via interface SPI, responsável pela transmissão dos pacotes de dados para o Gateway.
- **Gerenciamento de Energia:** O sistema é alimentado por uma bateria de Polímero de Lítio (Li-Po) de 3.7V, integrada a um módulo conversor de tensão dedicado.

2.1.1 Montagem e Conexões do Nó Sensor

A integração física dos componentes do nó sensor foi realizada em placa perfurada universal, adotando-se o uso de barras de pinos fêmeas para o acoplamento dos módulos. Essa abordagem permite que os componentes sejam retirados, mas garante uma conexão robusta que evita o mau contato e a instabilidade comuns no uso de protoboards.



As conexões entre o microcontrolador STM32F103 (Blue Pill) e os componentes segue o esquemático da figura abaixo, e a configuração descrita:



1. Sensor de Temperatura e Umidade (AHT10): Utiliza o barramento de comunicação I²C 1 do microcontrolador:

- SCL (Clock): Conectado ao pino PB10.
- SDA (Dados): Conectado ao pino PB11.
- VCC/GND: Ligados ao barramento de alimentação da placa.

2. Transceptor de Rádio (nRF24L01): Utiliza a interface SPI 1 para alta velocidade de transferência:

- CE (Chip Enable): Conectado ao pino PA4.
- CSN (Chip Select): Conectado ao pino PA3.
- SCK (Clock SPI): Conectado ao pino PA5.
- MISO (Master In Slave Out): Conectado ao pino PA6.
- MOSI (Master Out Slave In): Conectado ao pino PA7.

3. Sensor de Luminosidade (LDR): Configurado como um divisor de tensão resistivo:

- Sinal Analógico: Um pino LDR é conectado ao pino PA0 (ADC0), enquanto outro pino é conectado ao resistor, que leva ao GND.

4. Sistema de Alimentação:

- Entrada de Energia: A saída regulada do conversor DC-DC é conectada diretamente aos pinos OUT(+) -> VBAT e OUT(-) ->GND do STM32, e as Entradas são conectadas à bateria.

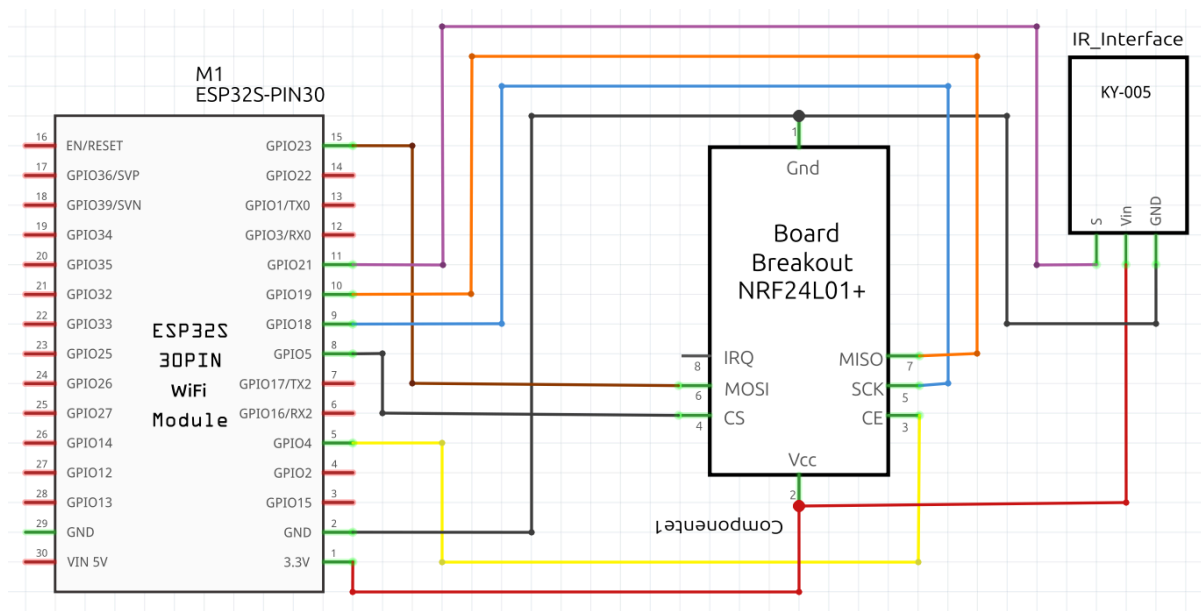
2.2 Gateway

O Gateway atua como o elemento central da topologia da rede, desempenhando as funções de recebimento de dados dos sensores e controle dos aparelhos de ar-condicionado. A sua arquitetura de hardware foi projetada para suportar conectividade híbrida (Radiofrequência e Wi-Fi) e processamento de protocolos de rede. O gateway é composto pelos seguintes componentes:

- **Microcontrolador ESP32:** Responsável pelo processamento central e conectividade, gerenciando a comunicação Wi-Fi com a nuvem (via MQTT/HTTP) e a coordenação da rede local;
- **Transceptor nRF24L01:** Módulo de rádio frequência operando em 2.4 GHz, conectado via interface SPI, responsável pelo recebimento dos pacotes de dados enviados pelos nós sensores;
- **Emissor Infravermelho (LED IR):** Atuador conectado a uma saída digital (GPIO) do microcontrolador, utilizado para enviar comandos de controle (ligar/desligar, temperatura) aos aparelhos de ar-condicionado;
- **Gerenciamento de Energia:** O sistema é alimentado pela rede elétrica, devido a alta demanda de energia para controle de dados.

2.2.1 Montagem e Conexões do Gateway

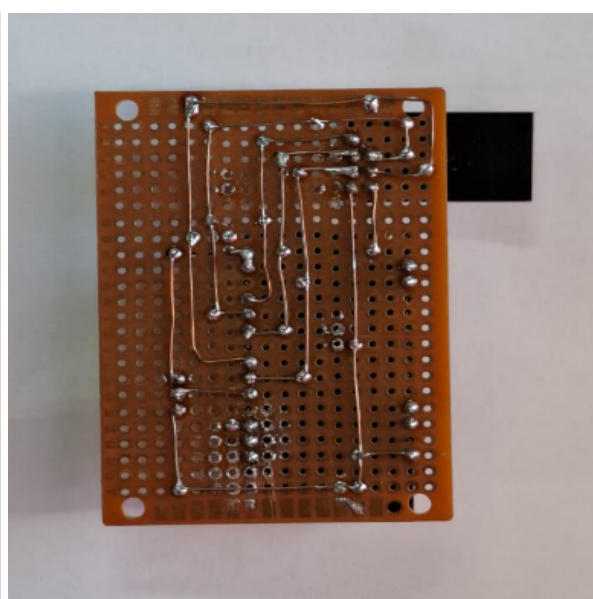
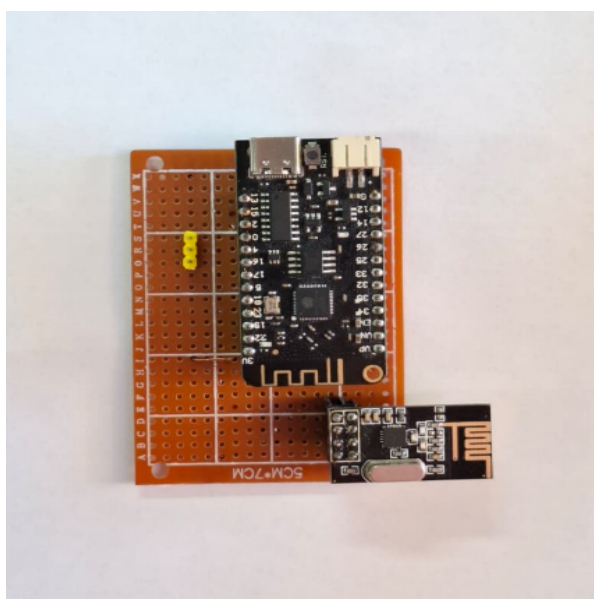
As conexões entre o microcontrolador, o módulo de rádio e o IR, foram feitas seguindo o esquemático abaixo.



A montagem física do gateway seguiu os mesmos parâmetros que a montagem física dos nós, foi utilizado uma placa perfurada universal e adotou-se o uso de barra de pinos fêmeas.

A transição do projeto lógico para o físico priorizou a estabilidade das conexões e a modularidade do sistema:

- **Modularidade:** O uso de barras de pinos fêmeas permite a remoção do ESP32 e do nRF24L01 para testes ou substituição.
- **Conexões:** No verso da placa, foram moldadas trilhas com fios de cobre para garantir a continuidade elétrica e organizar o barramento.
- **Acabamento:** A soldagem direta na placa perfurada assegura maior resistência mecânica comparada ao uso de protoboards.



1. Transceptor de Rádio (NRF24L01+) O módulo de rádio utiliza a interface VSPI padrão do ESP32 para comunicação:

- MOSI (Master Out Slave In): Conectado ao pino GPIO 23.
- MISO (Master In Slave Out): Conectado ao pino GPIO 19.
- SCK (Clock SPI): Conectado ao pino GPIO 18.
- CS/CSN (Chip Select): Conectado ao pino GPIO 5.
- CE (Chip Enable): Conectado ao pino GPIO 4.
- Alimentação: Pinos Vcc e Gnd conectados, respectivamente, ao VIN 5V e GND do ESP32 (utilizando o regulador interno da placa breakout).

2. Interface de Controle Infravermelho: Um módulo emissor IR pode ser controlado por um pino de saída digital para modulação do sinal:

- S (Sinal): Conectado ao pino GPIO 21.
- Alimentação: Pinos Vin e GND conectados ao barramento de alimentação da placa.

3. Desenvolvimento Firmware e Comunicação

3.1 Firmware Nó

O firmware do nó foi desenvolvido para o microcontrolador STM32F103 (Bluepill) utilizando programação bare-metal via acesso direto a registradores (CMSIS, *stm32f1xx.h*), sem HAL. A inicialização do clock do sistema é realizada manualmente em `SystemClock_Config()`, habilitando o HSE e configurando o PLL em 72 MHz. Para depuração e telemetria local, foi utilizada a USART2 em modo somente transmissão (TX), configurada em `USART2_Init_Manual()` para 115200 bps.

A aplicação segue um ciclo periódico:

- Inicializa clock, GPIO, UART, sensores e gerenciador de sleep.
- Configura o rádio NRF24L01+ no modo transmissor.
- Realiza leituras de sensores (temperatura/umidade e LDR).
- Empacota e transmite um payload fixo de 16 bytes via NRF24.
- Desliga rádio e periféricos, reconfigura pinos para baixo consumo.
- Entra em modo STOP por tempo definido via RTC Alarm.
- Ao acordar, reconfigura clock e reinicializa periféricos.

3.1.1 Aquisição de dados dos sensores

O nó coleta medições de **temperatura e umidade** via sensor **AHT10**, conectado ao barramento **I2C2** (pinos PB10/PB11) já a medição de **Luminosidade** via LDR conectado ao ADC(PB0), obtida como valor bruto e posteriormente escalonada. A configuração de hardware dos pinos ocorre em `GPIO_Init_Manual()`:

- I2C2 em **AF Open-Drain 2 MHz** em PB10/PB11.
- Entrada analógica para ADC em PB0.
- UART2 TX em PA2 (AF push-pull).

A leitura do AHT10 ocorre por chamadas de funções a `aht10_init()` para inicialização do sensor e `aht10_read(&sensor_data)` para obter os dados da `sensor_data.temperature` e `sensor_data.humidity`. Já a leitura do LDR é

feita em duas etapas a `ldr_adc_value = ldr_read_raw()` onde recebe do valor do ADC.

A estrutura de dados enviada pelo nó é um payload binário fixo de 16 bytes, pensado para simplificar o tratamento tanto no nó quanto no gateway, em vez de enviar texto (ASCII), o nó serializa diretamente uma struct C em memória e transmite esses bytes “crus” pelo NRF24. Isso reduz overhead, acelera o envio e evita parsing no gateway.

O formato de dados transmitido é:

```
typedef struct __attribute__((__packed__)) {  
    uint8_t  board_id;  
    float    temperature;  
    float    humidity;  
    uint16_t adc_raw;  
} SensorData_t;
```

Esse struct ocupa 11 bytes, portanto o firmware cria um buffer de 16 bytes e completa o restante com zeros (padding manual), mantendo o payload sempre com o mesmo tamanho:

```
/* payload fixo 16 bytes */  
uint8_t pkt[16];  
memset(pkt, 0, sizeof(pkt));  
memcpy(pkt, &tx_data, sizeof(tx_data));
```

A transmissão ocorre com a função `Nrf24_send(&radio, pkt)` e a confirmação por `Nrf24_isSend(&radio, 50)` com timeout.

A rotina de configuração do rádio (`NRF_Gateway_TX_Setup()`) define:

- **Canal RF (0x5A):** fixa nó e gateway na mesma frequência lógica, garantindo compatibilidade e reduzindo interferências/canais concorrentes.
- **Payload (16 bytes):** tamanho fixo para simplificar serialização/leitura e manter consistência do pacote (com *padding* quando necessário).
- **Endereço (0xE7E7E7E7E7, pipe 0):** identifica o enlace; o nó usa TX_ADDR e RX_ADDR_P0 (para ACK) e o gateway escuta no pipe 0.
- **Data rate (250 kbps):** prioriza robustez e alcance em ambientes com ruído, aumentando tolerância a perdas.
- **Potência (PA_MAX):** maximiza a confiabilidade do enlace.
- **Auto-ACK (EN_AA=0x01):** o gateway confirma a recepção automaticamente, permitindo detecção de falhas e retransmissão pelo hardware.

3.1.2 Habilitação do modo standby e wake-up por RTC

Para maximizar a autonomia energética, o nó entra em baixo consumo após cada ciclo de aquisição e transmissão. Esse mecanismo é baseado no modo **STOP** do STM32F103, com **wake-up pelo RTC Alarm** usando o oscilador **LSI**, sendo controlado pelo módulo `sleepmgr`. A ideia é reduzir o tempo em operação plena,

mantendo o microcontrolador dormindo na maior parte do período e acordando apenas para ler sensores, transmitir e voltar a dormir.

Antes de entrar em STOP, o firmware realiza uma sequência de “desligamento seguro” dos periféricos, principalmente do enlace de rádio. Primeiramente, o NRF24 é colocado em **power-down** via `nrf24_power_down(&radio)`, garantindo CE em nível baixo e escrevendo o registrador CONFIG com `PWR_UP=0`, o que desativa o circuito interno do transceptor e reduz significativamente o consumo. Em seguida, `nrf24_pins_safe()` força CE baixo e CSN alto, evitando seleção indevida do rádio durante o período de sono.

Além disso, o barramento SPI é desabilitado e seus pinos são colocados em um estado de menor consumo. A função `gpio_lowpower_spi_pins()` desativa o SPI1 e configura os pinos PA5/PA6/PA7 (SCK/MISO/MOSI) como entrada analógica, reduzindo correntes de fuga e impedindo que o microcontrolador mantenha *drivers* ativos desnecessariamente. Após essa preparação, o nó entra em STOP por um intervalo fixo usando `sleepmgr_sleep_seconds(60)`, permanecendo em baixo consumo até que o RTC dispare o alarme.

Quando ocorre o wake-up pelo RTC Alarm, o firmware precisa restaurar o ambiente de execução, pois após o STOP o microcontrolador tipicamente retorna para clock interno (HSI) e parte do estado dos periféricos pode ser perdido. Essa restauração é feita em `after_stop_reinit_everything()`, que reconfigura GPIOs essenciais, reinicializa a USART2, reativa os sensores por meio da rotina `sensors_reinit_after_stop()`, restaura os pinos CSN/CE do NRF24, reabilita o clock e a configuração do SPI1, e finalmente executa novamente a configuração completa do rádio em `NRF_Gateway_TX_Setup()`. Esse procedimento garante que, a cada retorno do STOP, o sistema volte a operar de forma determinística e consistente para o próximo ciclo de medição e transmissão.

3.2. Firmware Gateway

O firmware do Gateway foi desenvolvido sobre o framework ESP-IDF, utilizando o sistema operacional de tempo real FreeRTOS para gerenciamento de tarefas e recursos. A arquitetura explora o processador *dual-core* do ESP32 para implementando duas tasks rodando em núcleos diferentes e responsáveis pela aquisição de dados e o envio para a nuvem. As responsabilidades foram divididas da seguinte forma:

- **Tarefa de Recepção (Core 0 - Tempo Real):** A tarefa `nrf24_rx_task` foi alocada exclusivamente no **Core 0**. Sua função é realizar o *polling* do rádio NRF24L01+ com alta prioridade. Ao validar um pacote recebido, ela insere os dados brutos imediatamente em uma fila, liberando o processador para a próxima leitura. Isso isola o driver de rádio de qualquer bloqueio causado pela pilha TCP/IP.
- **Fila de Sincronização (Queue):** A comunicação entre os núcleos ocorre via `QueueHandle_t` (fila do FreeRTOS). Esta estrutura atua como um *buffer* circular, não vinculado a um núcleo, permitindo que a recepção de dados continue ocorrendo mesmo que a tarefa de envio esteja momentaneamente travada aguardando resposta do servidor.
- **Tarefa de Processamento e Nuvem (Core 1 - Aplicação):** A tarefa `influx_task` roda no **Core 1**. Ela atua como "consumidora", retirando os

dados da fila, convertendo as grandezas físicas (cálculo de ponto flutuante) e gerenciando a conexão HTTP com o InfluxDB. Caso ocorra lentidão ou queda no Wi-Fi, apenas esta tarefa é afetada, mantendo a coleta de sensores no Core 0 funcionando.

```
===== NRF Configuration =====
STATUS          = 0x0e RX_DR=0 TX_DS=0 MAX_RT=0 RX_P_NO=7 TX_FULL=0
RX_ADDR_P0-1    = 0xe7e7e7e7e7 0xe7e7e7e7e7
RX_ADDR_P2-5    = 0xc3 0xc4 0xc5 0xc6
TX_ADDR         = 0xe7e7e7e7e7
RX_PW_P0-6      = 0x10 0x10 0x00 0x00 0x00 0x00
EN_AA           = 0x3f
EN_RXADDR        = 0x03
RF_CH           = 0x5a
RF_SETUP        = 0x27
CONFIG          = 0x2b
DYNPD/FEATURE    = 0x00 0x00
Data Rate       = 250Kbps
CRC Length      = 8 bits
PA Power        = PA_MAX
Retransmit      = 2750 us
```

3.3 Comunicação Entre os Módulos

A interação com o transceptor nRF24L01+ é intermediada por uma biblioteca de driver de baixo nível, identificada como `mirf.h`. Esta biblioteca encapsula a complexidade do protocolo SPI e o mapa de registradores do chip, permitindo que o firmware configure o hardware através de uma estrutura de dispositivo (`NRF24_t dev`).

Para garantir a estabilidade do sinal e o alcance necessário, os parâmetros elétricos do rádio são definidos explicitamente na função de inicialização da tarefa:

```
Nrf24_init(&dev);

Nrf24_SetOutputRF_PWR(&dev, 3);
Nrf24_SetSpeedDataRates(&dev, 2);
Nrf24_setRetransmitDelay(&dev, 10);
Nrf24_setRetransmitCount(&dev, 10);
```

O processo de recepção segue três passos principais:

1. **Monitoramento (Polling):** A tarefa verifica continuamente o registrador de status do rádio para detectar a chegada de novos pacotes, garantindo resposta imediata sem depender de interrupções.
2. **Leitura e Conversão:** Ao identificar dados disponíveis, o firmware baixa os 16 bytes brutos via SPI e os reconstrói (*deserializa*) para a estrutura original (`SensorData_t`), recuperando o ID do nó e os valores de temperatura/umidade.

3. **Timestamping:** Antes de enviar o pacote para a fila de processamento, o sistema anexa o horário atual (*timestamp*), garantindo que o dado tenha um registro temporal preciso de quando chegou ao Gateway.

Ao receber um pacote válido do rádio, a tarefa tenta inseri-lo na fila utilizando a função do FreeRTOS `xQueueSend()` com o parâmetro de tempo de espera definido como zero. Isso instrui o escalonador do sistema operacional a realizar uma tentativa imediata de escrita. Se a fila tiver espaço, o dado é copiado e a tarefa volta a monitorar o rádio instantaneamente. Se a fila estiver cheia, a função retorna um erro imediatamente, em vez de colocar a tarefa em estado de espera (*Blocked*). Essa decisão de projeto impede que a tarefa de rádio fique travada aguardando a liberação de memória, o que poderia causar a perda de pacotes RF subsequentes.

```
if (xQueueSend(g_sample_q, &smp, 0) != pdTRUE)
{
    Sample_t dummy;
    (void)xQueueReceive(g_sample_q, &dummy, 0);
    (void)xQueueSend(g_sample_q, &smp, 0);
    ESP_LOGW(TAG, "Fila cheia: sobrescrevendo (ID=%u)", rx.board_id);
}
```

3.4 Comunicação com Influx

O gateway atua como um cliente HTTP que envia dados de sensores para o InfluxDB Cloud utilizando a API v2 de escrita. Após estabelecer conexão Wi-Fi, ele monta uma URL no formato `/api/v2/write?org=<ORG>&bucket=<BUCKET>&precision=s`, definindo explicitamente a organização, o bucket de destino e a precisão do timestamp em segundos. Cada amostra recebida do NRF24 é convertida para o Line Protocol, que inclui: nome da medição (`wsn`), tags (por exemplo, `node=<id>` para identificar o nó), campos numéricos (`t`, `h`, `l`) e um timestamp Unix. Esses dados são agrupados em um buffer local e enviados em lote por meio de requisições POST via `esp_http_client`, com cabeçalho `Content-Type: text/plain`.

A autenticação com o InfluxDB é feita usando um token de API, enviado no cabeçalho `HTTP Authorization: Token <token>`, garantindo que apenas clientes autorizados consigam gravar dados. O gateway implementa lógica de *batching* e controle de fluxo: ele só tenta enviar dados quando o Wi-Fi está conectado, envia automaticamente quando atinge um número máximo de pontos ou tamanho de buffer, e também força o envio após um tempo de espera, mesmo com poucos dados. Isso reduz o número de requisições, evita perda de dados quando a fila cresce e mantém o uso dentro dos limites de escrita do plano gratuito do InfluxDB Cloud.

3.5 Atuador

Acerca do atuador foi desenvolvido uma biblioteca `ir_tx.h`, essa biblioteca implementa uma camada de transmissão infravermelha (IR) para ESP32 utilizando o periférico RMT (Remote Control) do ESP-IDF, com foco em reproduzir fielmente sinais IR previamente capturados (ON e OFF) aplicando uma portadora típica de 38 kHz. O código começa definindo parâmetros padrão de temporização e modulação — como resolução de 1 μ s, frequência da portadora, duty cycle e tempo de gap

entre quadros — e armazena dois conjuntos grandes de símbolos (RX_ON_SIG e RX_OFF_SIG) que representam a forma de onda original recebida (pares mark/space em microssegundos). Esses símbolos são então convertidos para o formato de transmissão (s_tx_on e s_tx_off) pela função `build_tx_from_rx`, que inverte a semântica de nível para o transmissor: nível alto (carrier ativa) durante o “mark” e nível baixo (carrier desligada) durante o “space”, garantindo compatibilidade com a modulação IR.

A inicialização ocorre em `ir_tx_init_default`, que cria e configura o canal RMT de transmissão no GPIO indicado, aplica a portadora configurada, instancia um copy encoder (adequado quando os símbolos já estão pré-formatados) e habilita o canal. A função também garante que a inicialização seja idempotente e registra parâmetros relevantes para depuração. O controle da portadora é encapsulado em `apply_carrier`, permitindo alterar dinamicamente frequência e duty cycle via `ir_tx_set_carrier`, enquanto `ir_tx_set_gap_us` ajusta o intervalo entre transmissões. O envio propriamente dito é feito por `send_symbols`, que transmite o buffer de símbolos pelo RMT, bloqueia até o término da transmissão e aplica um atraso final (gap) para respeitar o protocolo temporal do dispositivo IR alvo. Por fim, `ir_tx_send_on` e `ir_tx_send_off` expõem uma API simples para disparar os quadros ON e OFF, abstraindo toda a complexidade do RMT e garantindo temporização precisa e repetível do sinal

4. Armazenamento e Visualização de Dados

Para garantir a persistência e análise histórica das variáveis monitoradas, foi utilizada uma *stack* de software composta por InfluxDB e Grafana.

- **InfluxDB:** Banco de dados de séries temporais (*Time Series Database*), escolhido por sua alta performance na gravação de dados de sensores.
- **Grafana:** Plataforma de visualização conectada ao InfluxDB, onde foram configurados *dashboards* para monitoramento em tempo real da temperatura interna, umidade e níveis de luminosidade.

link do dashboard:

<https://arthursnoww.grafana.net/public-dashboards/b06b86c9894d44b6b17ec7a07fc9acc1>

2026-01-21 13:24:14 GMT-3	n03	26,81	60,82	93,43
2026-01-21 13:24:14 GMT-3	n05	26,71	61,38	84,34
2026-01-21 13:24:17 GMT-3	n02	26,74	60,25	94,85
2026-01-21 13:24:24 GMT-3	n06	26,55	60,73	92,12
2026-01-21 13:24:32 GMT-3	n04	26,63	61,71	87,58
2026-01-21 13:25:03 GMT-3	n01	26,61	60,96	93,23



5. Resultados

O projeto apresentou o desenvolvimento e a implementação da Rede de Sensores Sem Fio (WSN) para o monitoramento ambiental de salas de aula com sucesso. A arquitetura proposta permitiu a coleta estável de dados (temperatura, umidade e luminosidade) através de nós sensores no microcontrolador STM32.

A integração entre o Gateway (ESP32) e a stack de dados (InfluxDB e Grafana) permitiu a visualização em tempo real das condições do ambiente, cumprindo o objetivo de fornecer dados para a otimização do conforto térmico. Além disso, o desenvolvimento da interface de controle por infravermelho (IrDA) validou a capacidade do sistema em atuar diretamente sobre os aparelhos de ar-condicionado, fechando o ciclo de automação voltado à eficiência energética.

Os testes realizados demonstraram que: **Eficiência Energética:** O uso de rotinas de Deep Sleep e o gerenciamento seguro de periféricos (como o desligamento do rádio nRF24L01) garantiram a autonomia necessária para os nós operarem por bateria. **Confiabilidade de Dados:** A estratégia de comunicação via FreeRTOS no Gateway, separando a recepção de rádio (Core 0) do processamento em nuvem (Core 1), evitou a perda de pacotes mesmo durante instabilidades na rede Wi-Fi. **Robustez Física:** A montagem em placas perfuradas com trilhas de cobre superou a instabilidade comum de protoboards, resultando em um hardware apto para o uso contínuo em ambiente real.

O repositório do projeto pode ser encontrado na seguinte URL: <https://github.com/Arthursnoww/wsn-environment-monitoring>, o qual possui *branches* com nomes auto-explicativos: node (com os arquivos dos nós da STM32CubeIDE), gateway (com os arquivos de FreeRTOS da ESP32), images (imagens dos módulos, circuitos, entre outros) e csv (com os dados de 24h contínuas de leitura).